JOBSHEET 19

IMPLEMENTASI UNIT TESTING PADA NEXT.JS MENGGUNAKAN JEST

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_19

PRAKTIKUM 1 – Setup Jest di Next.js

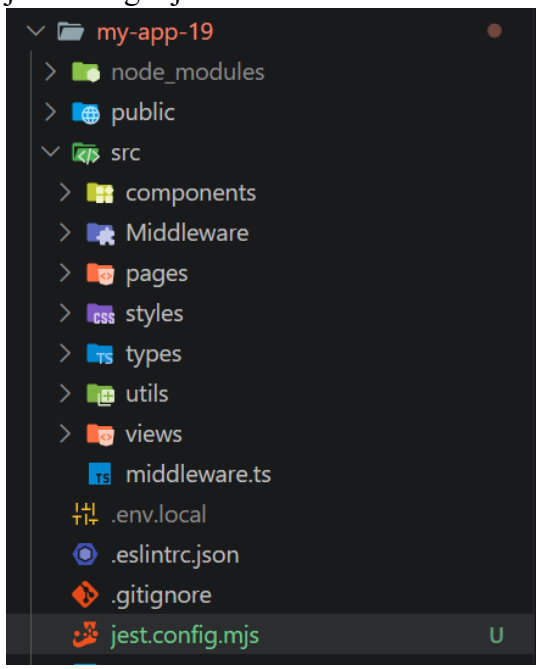
1. Install Dependencies

- Jalankan:
- npm install jest jest-environment-jsdom @testing-library/react @testing- library/jest-dom --save-dev –force

```
PS D:\Pemograman_Framework\praktikum_19\my-app-19> npm install jest jest-environment-jsdom @testing-library/react @testing-library/jest-dom --save-dev --force
npm WARN deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the curren
t version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm WARN deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the curre
nt version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 290 packages, and audited 740 packages in 12s

184 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

2. Buat File Konfigurasi
   Doc : https://nextjs.org/docs/pages/guides/testing/jest

- Buat file:
  jest.config.mjs



- Isi dengan:

```jest.config.mjs
import nextJest from 'next/jest.js';

const createJestConfig = nextJest({
  dir: './',
});

const config = {
    coverageProvider: 'v8',
    testEnvironment: 'jsdom',
}

export default createJestConfig(config);
```

- Tambahkan Script di package.json

```package.json
jest.config.mjs U    package.json M
my-app-19 > package.json > ...
{
  "name": "my-app",
  "version": "0.1.0",
  "private": true,
  ▷Debug
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "test": "jest --passWithNoTests -u",
    "test:coverage": "npm run test -- --coverage",
    "test:watch": "jest --watchAll"
  },
```

PRAKTIKUM 2 – Struktur Folder Testing

Buat folder:

src/__test__/

Struktur contoh:

```
src
├── pages
```

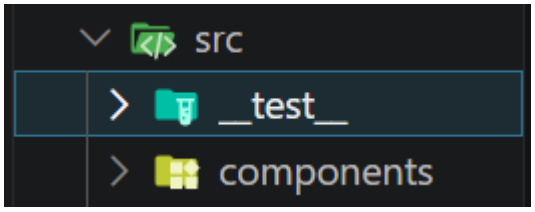
├── components

├── views

└── __test__

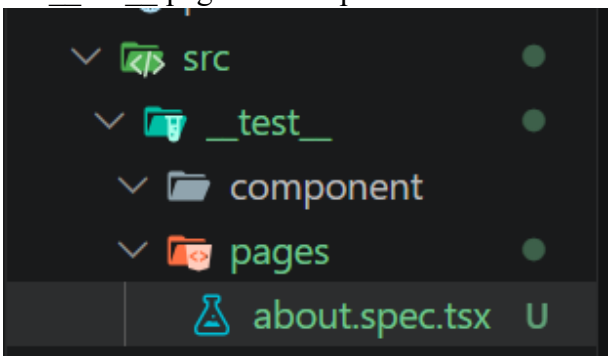
├── pages

└── components



PRAKTIKUM 3 – Testing Halaman About

1. Buat File Testing
   src/__test__/pages/about.spec.tsx



2. Contoh Testing Snapshot. Pada about.spec.tsx tambahkan code berikut :

```
import { render } from '@testing-library/react';
import AboutPage from '@/pages/about';

describe('About Page', () => {
    it('renders about page correctly', () => {
        const page = render(<AboutPage />)
        expect(page).toMatchSnapshot();
    })
})

```

3. Jalankan Testing

- npm run test
- Jika berhasil:
- PASS about.spec.tsx

```
PS D:\Pemograman_Framework\praktikum_19\my-app-19> npm run test

> my-app@0.1.0 test
> jest --passWithNoTests -u

 PASS  src/__test__/pages/about.spec.tsx
  About Page
    √ renders about page correctly (39 ms)

 › 1 snapshot written.
Snapshot Summary
 › 1 snapshot written from 1 test suite.
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   1 written, 1 total
Time:        1.316 s
Ran all test suites.
PS D:\Pemograman_Framework\praktikum_19\my-app-19>
```

PRAKTIKUM 4 – Coverage Report

- Jalankan:
  npm run test:coverage

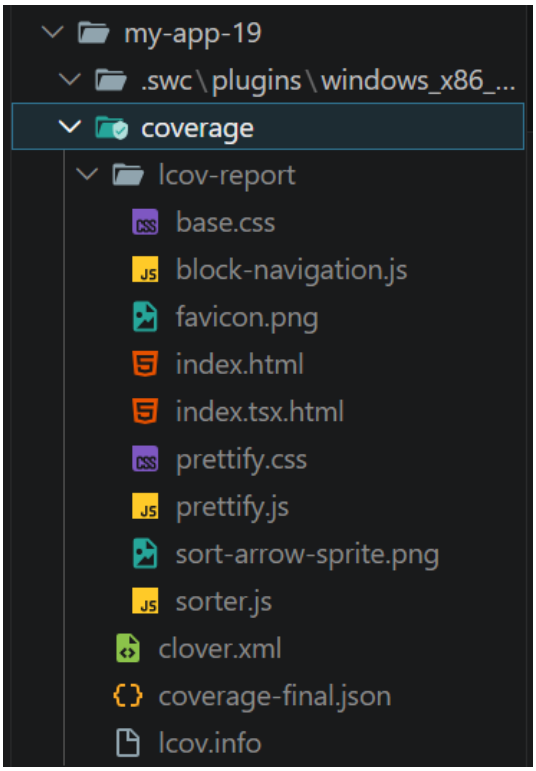
```
PS D:\Pemograman_Framework\praktikum_19\my-app-19> npm run test:coverage

> my-app@0.1.0 test:coverage
> npm run test -- --coverage


> my-app@0.1.0 test
> jest --passWithNoTests -u --coverage

 PASS  src/__test__/pages/about.spec.tsx
  About Page
    √ renders about page correctly (35 ms)

-----------|---------|----------|---------|---------|-------------------
File       | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----------|---------|----------|---------|---------|-------------------
All files  |     100 |      100 |     100 |     100 |
 index.tsx |     100 |      100 |     100 |     100 |
-----------|---------|----------|---------|---------|-------------------
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   1 passed, 1 total
Time:        1.65 s, estimated 2 s
Ran all test suites.
```

- Akan muncul folder:

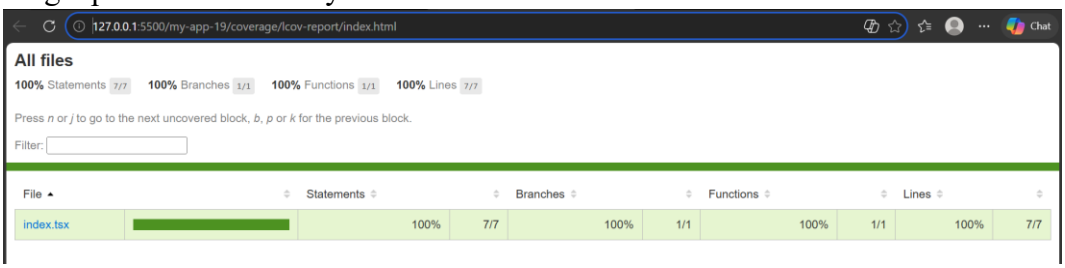
  coverage/

  Buka:

  coverage/lcov-report/index.html (buka di melalui explorer)



  Target perusahaan biasanya:



  Minimum 80% coverage

PRAKTIKUM 5 – Konfigurasi Coverage Lengkap

- Update jest.config.mjs:

```
import nextJest from 'next/jest.js'

const createJestConfig = nextJest({
  dir: './',
})

const config = {
  testEnvironment: "jsdom",
  modulePaths: ['<rootDir>/src/'],
  collectCoverage: true,
  collectCoverageFrom: [
    '**/*.{ts,tsx}',
    '**/*.d.ts',
    '!**/node_modules/**',
    '!**/.next/**',
    '!**/coverage/**',
    '!**/jest.config.mjs',
    '!**/next.config.mjs',
    '!**/types/**',
    '!**/views/**',
    '!**/pages/api/**',
  ],
}

export default createJestConfig(config)

```

- Jalankan npm run test:coverage

```
PS D:\Pemograman_Framework\praktikum_19\my-app-19> npm run test:coverage

 > my-app@0.1.0 test:coverage
 > npm run test -- --coverage


 > my-app@0.1.0 test
 > jest --passWithNoTests -u --coverage

 PASS  src/__test__/pages/about.spec.tsx
   About Page
     √ renders about page correctly (36 ms)
```

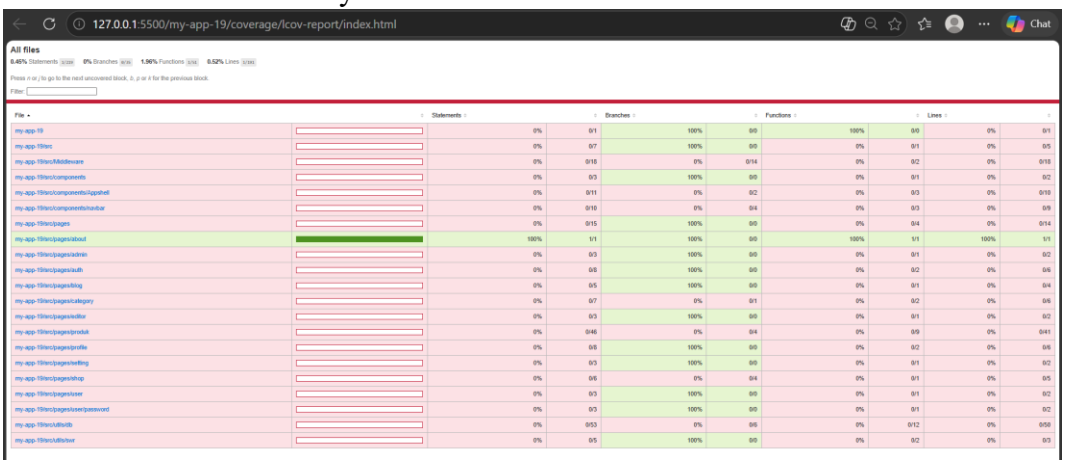
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	0.45	0	1.96	0.52	
my-app-19	0	100	100	0	
next-env.d.ts	0	100	100	0	3
my-app-19/src	0	100	0	0	
middleware.ts	0	100	0	0	1-11
my-app-19/src/Middleware	0	0	0	0	
withAuth.ts	0	0	0	0	1-31
my-app-19/src/components	0	100	0	0	
index.tsx	0	100	0	0	1-9
my-app-19/src/components/Appshell	0	0	0	0	
index.tsx	0	0	0	0	1-35
my-app-19/src/components/navbar	0	0	0	0	
index.tsx	0	0	0	0	1-67
my-app-19/src/pages	0	100	0	0	
404.tsx	0	100	0	0	1-55
_app.tsx	0	100	0	0	1-8
_document.tsx	0	100	0	0	1-3
index.tsx	0	100	0	0	2-11
my-app-19/src/pages/about	100	100	100	100	
index.tsx	100	100	100	100	
my-app-19/src/pages/admin	0	100	0	0	
index.tsx	0	100	0	0	1-18
my-app-19/src/pages/auth	0	100	0	0	
login.tsx	0	100	0	0	1-12
register.tsx	0	100	0	0	2-12
my-app-19/src/pages/blog	0	100	0	0	
[slug].tsx	0	100	0	0	1-14
my-app-19/src/pages/category	0	0	0	0	
[...slug].tsx	0	0	0	0	1-24
my-app-19/src/pages/editor	0	100	0	0	
index.tsx	0	100	0	0	1-16

```
PS D:\Pemograman_Framework\praktikum_19\my-app-19> npm run test:coverage
```

my-app-19/src/pages/auth	0	100	0	0	
login.tsx	0	100	0	0	1-12
register.tsx	0	100	0	0	2-12
my-app-19/src/pages/blog	0	100	0	0	
[slug].tsx	0	100	0	0	1-14
my-app-19/src/pages/category	0	0	0	0	
[...slug].tsx	0	0	0	0	1-24
my-app-19/src/pages/editor	0	100	0	0	
index.tsx	0	100	0	0	1-16
my-app-19/src/pages/produk	0	0	0	0	
[produk].tsx	0	100	0	0	4-69
index.tsx	0	0	0	0	1-23
server.tsx	0	100	0	0	1-23
static.tsx	0	100	0	0	1-21
my-app-19/src/pages/profile	0	100	0	0	
edit.tsx	0	100	0	0	1-9
index.tsx	0	100	0	0	1-13
my-app-19/src/pages/setting	0	100	0	0	
app.tsx	0	100	0	0	1-9
my-app-19/src/pages/shop	0	0	0	0	
[[...slug]].tsx	0	0	0	0	1-22
my-app-19/src/pages/user	0	100	0	0	
index.tsx	0	100	0	0	1-9
my-app-19/src/pages/user/password	0	100	0	0	
index.tsx	0	100	0	0	1-9
my-app-19/src/utils/db	0	0	0	0	
firebase.ts	0	100	100	0	2-20
servicefirebase.ts	0	0	0	0	11-129
my-app-19/src/utils/swr	0	100	0	0	
fetcher.ts	0	100	0	0	1-4

```
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   1 passed, 1 total
Time:        4.7 s
Ran all test suites.
```

- Jika dilihat di index.htmlnya



PRAKTIKUM 6 – Testing dengan getByTestId

1. Tambahkan pada About Page
   <h1 data-testid="title">About Page</h1>

```jsx
const AboutPage = () => {
  return (
    <div>
      <h1 data-testid="title">About Page</h1>
    </div>
  );
}

export default AboutPage;
```

2. Update Testing pada about.spec.tsx

```
import { render, screen } from '@testing-library/react';
import AboutPage from '@/pages/about';

describe('About Page', () => {
    it('renders about page correctly', () => {
        const page = render(<AboutPage />)
        expect(screen.getByTestId("title").textContent).toBe("About Page");
        expect(page).toMatchSnapshot();
    })
})
```

- Dicoba untuk di run

```
…
 ...shop |       0 |       0 |       0 |       0 |
  ...tsx |       0 |       0 |       0 |       0 | 1-22
 ...user |       0 |     100 |       0 |       0 |
  ...tsx |       0 |     100 |       0 |       0 | 1-9
 ...word |       0 |     100 |       0 |       0 |
  ...tsx |       0 |     100 |       0 |       0 | 1-9
 ...s/db |       0 |       0 |       0 |       0 |
  ....ts |       0 |     100 |     100 |       0 | 2-20
  ....ts |       0 |       0 |       0 |       0 | 11-129
 .../swr |       0 |     100 |       0 |       0 |
  ....ts |       0 |     100 |       0 |       0 | 1-4
---------|---------|---------|---------|---------|-------------------
Snapshot Summary
 › 1 snapshot updated from 1 test suite.

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   1 updated, 1 total
Time:        2.812 s
Ran all test suites.
```

- Coba Jika dibuat Salah:
  Rubah menjadi toBe("About")

```
import { render, screen } from '@testing-library/react';
import AboutPage from '@/pages/about';

describe('About Page', () => {
    it('renders about page correctly', () => {
        const page = render(<AboutPage />)
        expect(screen.getByTestId("title").textContent).toBe("About");
        expect(page).toMatchSnapshot();
    })
})
```

- Jalan kan dan Hasil:

FAIL

Expected: "About"

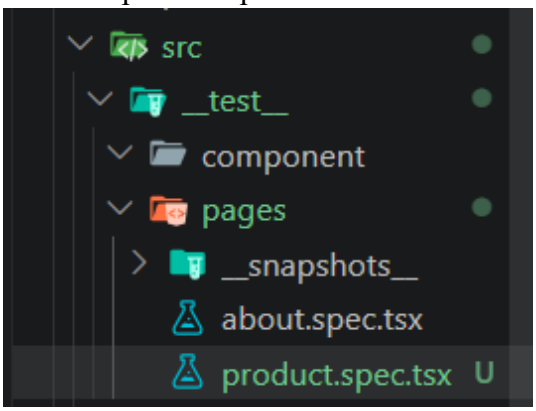
Received: "About Page"

```
 ...tsx |       0 |      100 |       0 |       0 | 1-9
...shop |       0 |        0 |       0 |       0 |
 ...tsx |       0 |        0 |       0 |       0 | 1-22
...user |       0 |      100 |       0 |       0 |
 ...tsx |       0 |      100 |       0 |       0 | 1-9
...word |       0 |      100 |       0 |       0 |
 ...tsx |       0 |      100 |       0 |       0 | 1-9
...s/db |       0 |        0 |       0 |       0 |
 ....ts |       0 |      100 |     100 |       0 | 2-20
 ....ts |       0 |        0 |       0 |       0 | 11-129
.../swr |       0 |      100 |       0 |       0 |
 ....ts |       0 |      100 |       0 |       0 | 1-4
---------|---------|----------|---------|---------|-------------------
Test Suites: 1 failed, 1 total
Tests:       1 failed, 1 total
Snapshots:   0 total
Time:        2.817 s
Ran all test suites.
```

PRAKTIKUM 7 – Testing Page dengan Router (Mocking)

Kita coba untuk melakukan testing pada halaman produk

1. Buat file product.spec.tsx



2. Tambahkan kode berikut

```
import { render, screen } from "@testing-library/react"
import TampilanProduk from "@/pages/produk"

describe("Product Page", () => {
  it("renders product page correctly", () => {
    const page = render(<TampilanProduk />)
    expect(screen.getByTestId("title").textContent).toBe("Product Page")
    expect(page).toMatchSnapshot()
  })
})

```

3. Ketika testing halaman Product, sering muncul error:

```
PASS  src/__test__/pages/about.spec.tsx
FAIL  src/__test__/pages/product.spec.tsx
  ● Product Page › renders product page correctly

    NextRouter was not mounted. https://nextjs.org/docs/messages/next-router-not-mounted

      6 |
      7 | const kategori = () => {
    > 8 |     const { push } = useRouter();
        |                      ^
      9 |     const [products, setProducts] = useState([]);
     10 |
     11 |

      at useRouter (node_modules/next/src/client/router.ts:141:11)
      at kategori (src/pages/produk/index.tsx:8:31)
      at Object.react_stack_bottom_frame (node_modules/react-dom/cjs/react-dom-client.development.js:25904:20)
      at renderWithHooks (node_modules/react-dom/cjs/react-dom-client.development.js:7662:22)
      at updateFunctionComponent (node_modules/react-dom/cjs/react-dom-client.development.js:10166:19)
      at beginWork (node_modules/react-dom/cjs/react-dom-client.development.js:11778:18)
      at runWithFiberInDEV (node_modules/react-dom/cjs/react-dom-client.development.js:874:13)
      at performUnitOfWork (node_modules/react-dom/cjs/react-dom-client.development.js:17641:22)
      at workLoopSync (node_modules/react-dom/cjs/react-dom-client.development.js:17469:41)
      at renderRootSync (node_modules/react-dom/cjs/react-dom-client.development.js:17450:11)
      at performWorkOnRoot (node_modules/react-dom/cjs/react-dom-client.development.js:16583:35)
      at performWorkOnRootViaSchedulerTask (node_modules/react-dom/cjs/react-dom-client.development.js:18957:7)
      at flushActQueue (node_modules/react/cjs/react.development.js:590:34)
      at process.env.NODE_ENV.exports.act (node_modules/react/cjs/react.development.js:884:10)
      at node_modules/@testing-library/react/dist/act-compat.js:46:25
      at renderRoot (node_modules/@testing-library/react/dist/pure.js:189:26)
      at render (node_modules/@testing-library/react/dist/pure.js:291:10)
      at Object.<anonymous> (src/__test__/pages/product.spec.tsx:6:24)

---------------------------------|---------|----------|---------|---------|-------------------
File                             | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
---------------------------------|---------|----------|---------|---------|-------------------
All files                        |    6.33 |        0 |    3.92 |    6.25 |
 my-app-19                       |       0 |      100 |     100 |       0 |
```

- NextRouter was not mounted
  Solusi: Mock Next Router
  Tambahkan di file product.spec.tsx

```
import { render, screen } from "@testing-library/react"
import TampilanProduk from "@/pages/produk"

jest.mock("next/router", () => ({
  useRouter() {
    return {
      route: "/product",
      pathname: "",
      query: {},
      asPath: "",
      push: jest.fn(),
      event: {
        on: jest.fn(),
        off: jest.fn(),
      },
      isReady: true,
    }
  },
}))

describe("Product Page", () => {
  it("renders product page correctly", () => {
    const page = render(<TampilanProduk />)
    expect(screen.getByTestId("title").textContent).toBe("Product Page")
    expect(page).toMatchSnapshot()
  })
})
```

PRAKTIKUM 8 – Menangani Undefined Data

- Jalankan npm run test:coverage maka akan muncul error
- Jika muncul error:
  Cannot read properties of undefined
  Perbaiki di komponen:
  Pada file index.tsx pada folder pages/produk

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";
import useSWR from "swr";
import fetcher from "../../utils/swr/fetcher";

const kategori = () => {
    const { push } = useRouter();
    const [products, setProducts] = useState([]);


    const { data, error, isLoading } = useSWR("/api/produk", fetcher);
    // cek apakah data, error, dan isLoading sudah bnar

    return (
        <div>
            <TampilanProduk
            products={isLoading ? [] : data?.data} />
        </div>
    );
};

export default kategori;
```

Jalankan npm run test:coverage maka akan muncul error
Maka Solusinya perbaiki code pada file src/views/produk/index.tsx

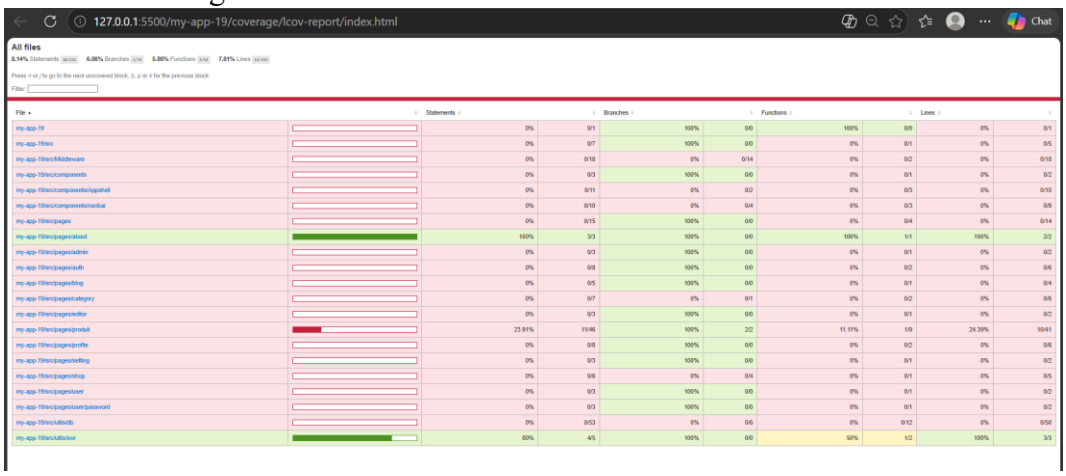
```
{products?.length > 0 ? (

  <>
  {products?.map((product: ProductType) => (
```

Note pastikan : comment pada code berikut pada 2 code testing

```python
import { render, screen } from "@testing-library/react"
import TampilanProduk from "@/pages/produk"

jest.mock("next/router", () => ({
  useRouter() {
    return {
      route: "/produk",
      pathname: "",
      query: {},
      asPath: "",
      push: jest.fn(),
      event: {
        on: jest.fn(),
        off: jest.fn(),
      },
      isReady: true,
    }
  },
}))

describe("Product Page", () => {
  it("renders product page correctly", () => {
    const page = render(<TampilanProduk />)
    // expect(screen.getByTestId("title").textContent).toBe("Product Page")
    expect(page).toMatchSnapshot()
  })
})
```

```python
import { render, screen } from '@testing-library/react';
import AboutPage from '@/pages/about';

describe('About Page', () => {
    it('renders about page correctly', () => {
        const page = render(<AboutPage />)
        // expect(screen.getByTestId("title").textContent).toBe("About Page");
        expect(page).toMatchSnapshot();
    })
})
```

Analisis Coverage



Jika dilihat aplikasi yang dibuat masih 5% jadi perlu banyak perbaikan sebelum proses deploy.
Perhatikan bagian:
Statement
Branch
Function
Lines

Contoh:

Metric	Hasil
Statements	85%
Branch	60%
Functions	90%
Lines	88%

Metric Hasil

Statements 85%

Branch 60%

Functions 90%

Lines 88%

Branch biasanya paling sulit karena perlu menguji kondisi if/else.

Standar Industri

Biasanya:

- ≥ 80% → boleh production
- < 80% → harus diperbaiki
- Semua critical feature wajib dites

Tugas Praktikum

1. Buat unit test untuk:

- Halaman Product
- 1 Komponen

```
import { render, screen } from "@testing-library/react"
import ProductPage from "@/pages/produk"

jest.mock("next/router", () => ({
  useRouter() {
    return {
      route: "/produk",
      pathname: "",
      query: {},
      asPath: "",
      push: jest.fn(),
      events: {
        on: jest.fn(),
        off: jest.fn(),
      },
      isReady: true,
    }
  },
}))

jest.mock("swr", () => ({
  __esModule: true,
  default: () => ({
    data: { data: [{ id: "1", name: "Produk A", price: 50000, image: "/img.jpg", category: "Elektronik" }] },
    error: null,
    isLoading: false,
  }),
}))

jest.mock("@/views/produk", () => ({
  __esModule: true,
  default: ({ products }: any) => (
    <div data-testid="product">
      {products.length} Produk
    </div>
  ),
}))

describe("Product Page", () => {

  it("renders product page correctly", () => {
    const page = render(<ProductPage />)
    expect(screen.getByTestId("product").textContent).toBe("1 Produk")
    expect(page).toMatchSnapshot()
  })

  it("renders dengan 0 produk jika data kosong", () => {
    // Override mock khusus test ini
    jest.mock("swr", () => ({
      __esModule: true,
      default: () => ({
        data: { data: [] },
        error: null,
        isLoading: false,
      }),
    }))
    const page = render(<ProductPage />)
    expect(page).toMatchSnapshot()
  })

})
```

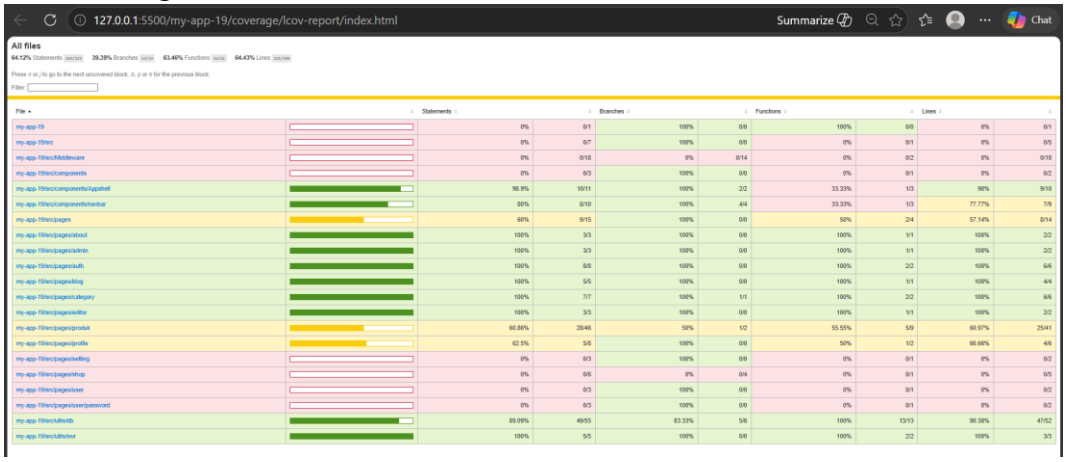
```
PASS  src/__test__/pages/product.spec.tsx
PASS  src/__test__/pages/about.spec.tsx
----|---------|----------|---------|---------|-------------------
... | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----|---------|----------|---------|---------|-------------------
... |    7.69 |     3.03 |    3.92 |    7.29 |
 ... |       0 |      100 |     100 |       0 |

  ... |       0 |      100 |     100 |       0 | 3

 ... |       0 |      100 |       0 |       0 |

  ... |       0 |      100 |       0 |       0 | 1-11

 ... |       0 |        0 |       0 |       0 |

  ... |       0 |        0 |       0 |       0 | 1-31

 ... |       0 |      100 |       0 |       0 |

  ... |       0 |      100 |       0 |       0 | 1-9

 ... |       0 |        0 |       0 |       0 |

  ... |       0 |        0 |       0 |       0 | 1-35
```

2. Gunakan minimal:

- 1 Snapshot test
- 1 toBe()
- 1 getByTestId()

```python
import { render, screen } from "@testing-library/react"
import ProductPage from "@/pages/produk"

jest.mock("next/router", () => ({
  useRouter() {
    return {
      route: "/produk",
      pathname: "",
      query: {},
      asPath: "",
      push: jest.fn(),
      events: {
        on: jest.fn(),
        off: jest.fn(),
      },
      isReady: true,
    }
  },
}))

jest.mock("swr", () => ({
  __esModule: true,
  default: () => ({
    data: { data: [{ id: "1", name: "Produk A", price: 50000, image: "/img.jpg", category: "Elektronik" }] },
    error: null,
    isLoading: false,
  }),
}))

jest.mock("@/views/produk", () => ({
  __esModule: true,
  default: ({ products }: any) => (
    <div data-testid="product">
      {products.length} Produk
    </div>
  ),
}))

describe("Product Page", () => {

  it("renders product page correctly", () => {
    const page = render(<ProductPage />)
    expect(screen.getByTestId("product").textContent).toBe("1 Produk")
    expect(page).toMatchSnapshot()
  })

  it("renders dengan 0 produk jika data kosong", () => {
    // Override mock khusus test ini
    jest.mock("swr", () => ({
      __esModule: true,
      default: () => ({
        data: { data: [] },
        error: null,
        isLoading: false,
      }),
    }))
    const page = render(<ProductPage />)
    expect(page).toMatchSnapshot()
  })

})
```

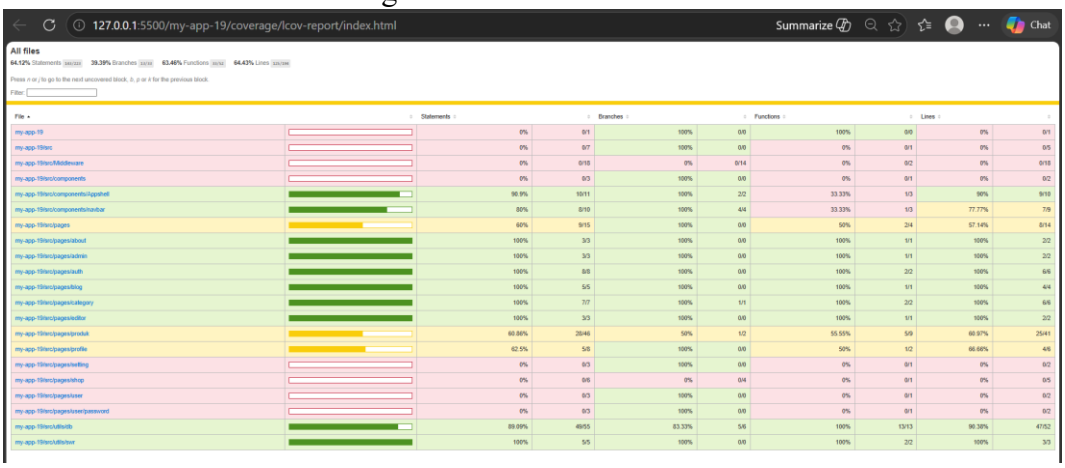
3. Buat coverage minimal 50%



4. Lakukan mocking untuk router

```
import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import TampilanLogin from "@/views/auth/login";
import "@testing-library/jest-dom";

jest.mock("next-auth/react", () => ({
  signIn: jest.fn(),
}));

jest.mock("next/router", () => ({
  useRouter: jest.fn(),
}));

import { signIn } from "next-auth/react";
import { useRouter } from "next/router";

describe("Login Page", () => {
  const pushMock = jest.fn();

  beforeEach(() => {
    jest.clearAllMocks();

    (useRouter as jest.Mock).mockReturnValue({
      push: pushMock,
      query: {},
    });
  });

  it("login success", async () => {
    (signIn as jest.Mock).mockResolvedValue({
      error: null,
    });

    render(<TampilanLogin />);

    fireEvent.submit(screen.getByRole("form"), {
      target: {
        email: { value: "test@mail.com" },
        password: { value: "123456" },
      },
    });

    await waitFor(() => {
      expect(signIn).toHaveBeenCalled();
      expect(pushMock).toHaveBeenCalled();
    });
  });

  it("login gagal", async () => {
    (signIn as jest.Mock).mockResolvedValue({
      error: "Login failed",
    });

    render(<TampilanLogin />);

    fireEvent.submit(screen.getByRole("form"), {
      target: {
        email: { value: "test@mail.com" },
        password: { value: "123" },
      },
    });

    await waitFor(() => {
      expect(screen.getByText("Login failed")).toBeInTheDocument();
    });
  });
});
```

5. Dokumentasikan hasil coverage



Diskusi & Refleksi

1. Mengapa unit testing penting sebelum production?
- Unit testing penting karena dapat memastikan setiap fungsi atau komponen berjalan sesuai yang diharapkan sebelum aplikasi digunakan oleh user. Dengan adanya testing, error dapat ditemukan lebih awal sehingga mengurangi risiko bug saat aplikasi sudah di production.

2. Mengapa branch coverage sulit mencapai 100%?
- Branch coverage sulit mencapai 100% karena setiap percabangan logika (seperti if-else atau switch) harus diuji semua kemungkinan kondisinya. Dalam praktiknya, ada beberapa kondisi yang jarang terjadi atau sulit disimulasikan, sehingga tidak semua cabang dapat diuji secara lengkap.

3. Apa itu mocking?
- Mocking adalah teknik dalam testing untuk mengganti fungsi atau modul asli dengan versi tiruan. Tujuannya agar proses testing tidak bergantung pada sistem luar, seperti API atau database, sehingga lebih cepat dan terkontrol.

4. Kapan snapshot test digunakan?
- Snapshot test digunakan untuk memastikan tampilan UI tidak berubah secara tidak sengaja. Biasanya dipakai pada komponen frontend untuk membandingkan hasil render saat ini dengan hasil sebelumnya.

5. Apakah semua file harus dites?
- Snapshot test digunakan untuk memastikan tampilan UI tidak berubah secara tidak sengaja. Biasanya dipakai pada komponen frontend untuk membandingkan hasil render saat ini dengan hasil sebelumnya.